

Relatório de Complexidade (Shell, Merge e Heap Sort)

Vitor K. Costa, Fernando Nunes e Arthur Abreu
Universidade do Vale do Itajaí (Univali) – Campus Kobrasol

16 de Junho de 2026

Resumo

Este relatório analisa o desempenho prático de três algoritmos de ordenação: *Shell Sort*, *Merge Sort* e *Heap Sort*. A avaliação foi feita medindo duas operações básicas: o número de Comparações $C(n)$ e a quantidade de Movimentações de dados $M(n)$. Os testes foram executados em diferentes tamanhos de entrada e em três situações: dados ordenados (crescente), dados invertidos (decrescente) e dados embaralhados (aleatório). Os resultados são apresentados em valores relativos percentuais para isolar qual algoritmo exige maior custo computacional em cada cenário.

1 Introdução

O estudo do desempenho de algoritmos de ordenação interna transcende a análise puramente teórica da notação Big- $\mathcal{O}$. Na prática, é essencial medir as operações estruturais que mais consomem processamento: as comparações $C(n)$ (avaliar chaves lógicas) e as movimentações $M(n)$ (troca física de elementos na memória).

- **Shell Sort:** É uma melhoria do *Insertion Sort* desenhada para realizar trocas de elementos distantes. A implementação deste documento utiliza a sequência original de intervalos de Hibbard/Shell $(n/2, n/4, \dots, 1)$. O defeito central dessa sequência ocorre em vetores estritamente invertidos, situação em que o algoritmo colapsa para a complexidade polinomial $\mathcal{O}(n^2)$, causando uma explosão no custo temporal e nas métricas de $M(n)$ e $C(n)$.
- **Merge Sort:** Otimizado pelo princípio de *divisão e conquista*, este método particiona recursivamente a estrutura de dados. Ele garante matematicamente o limite de tempo $\Theta(n \log n)$ em todas as permutações de entrada. O revés prático encontra-se na fase de intercalação (*merge*): ele é incapaz de operar *in-place* (no próprio espaço do vetor), exigindo a alocação de vetores auxiliares, o que eleva linearmente as movimentações na memória e gera penalidades de *cache miss*.
- **Heap Sort:** Estruturado em torno da lógica de *max-heap* (uma árvore binária onde nós pais mantêm valores superiores aos filhos). Possui limite garantido de $\mathcal{O}(n \log n)$ e executa a ordenação de forma *in-place*. Contudo, a necessidade de restaurar constantemente as propriedades da árvore (função *heapify*) exige baterias exaustivas de comparações lógicas, insuflando o valor total de $C(n)$.

2 Metodologia

As simulações leram arquivos de texto populados com 10.000 números inteiros. A coleta da variável $C(n)$ foi injetada no interior das estruturas condicionais e de repetição lógicas (e.g., $L[i] \leq R[j]$). A coleta da variável $M(n)$ foi aplicada sobre qualquer cópia, atribuição cruzada ou realocação em memória temporária.

Para estabelecer um parâmetro de comparação justo entre métodos com complexidades tão diferentes, calculou-se o **Valor Relativo**. Em cada simulação, a métrica mais ineficiente (o pior valor absoluto) converte-se na base (100%). O rendimento percentual dos demais métodos foi extraído pela fórmula:

$$V_{relativo} = \left(\frac{\text{Valor Atual}}{\text{Valor Máximo do Cenário}} \right) \times 100 \quad (1)$$

3 Resultados Empíricos

As tabelas expõem o percentual de ineficiência alcançado por cada bloco estrutural do código.

3.1 Cenário 1: Dados em Ordem Aleatória

Nesta configuração de dados embaralhados randomicamente, o teste evita instigar os piores cenários matemáticos das topologias e aproxima-se do uso convencional no mundo real.

Tabela 1: Valores Relativos (%) - Entrada Aleatória

Quantidade de dados (N)	Comparações $C(n)$			Movimentações $M(n)$		
	Shell Sort	Merge Sort	Heap Sort	Shell Sort	Merge Sort	Heap Sort
100	100,0	49,2	64,5	88,4	100,0	76,2
500	100,0	46,1	62,0	91,2	100,0	73,8
1000	100,0	48,3	61,1	90,0	100,0	75,0
10000	100,0	45,5	58,2	95,5	100,0	81,1

3.2 Cenário 2: Dados em Ordem Crescente (Melhor Caso)

O vetor entra em estado de pré-ordenação correta. O objetivo primário deste teste é atestar a capacidade da mecânica condicional de reconhecer a ordenação implícita e pular processos redundantes de rearranjo.

Tabela 2: Valores Relativos (%) - Entrada Crescente

Quantidade de dados (N)	Comparações $C(n)$			Movimentações $M(n)$		
	Shell Sort	Merge Sort	Heap Sort	Shell Sort	Merge Sort	Heap Sort
100	56,2	41,0	100,0	68,0	100,0	78,5
500	58,5	40,2	100,0	71,5	100,0	82,0
1000	58,3	41,3	100,0	70,1	100,0	80,1
10000	61,0	39,5	100,0	60,8	100,0	65,3

3.3 Cenário 3: Dados em Ordem Decrescente (Pior Caso)

A estrutura entra no sistema em ordem estritamente reversa, exigindo distâncias máximas de trânsito em memória para a recolocação dos índices. É o ambiente de prova fatal contra algoritmos sensíveis ao grau inicial de desordem.

Tabela 3: Valores Relativos (%) - Entrada Decrescente

Quantidade de dados (N)	Comparações $C(n)$			Movimentações $M(n)$		
	Shell Sort	Merge Sort	Heap Sort	Shell Sort	Merge Sort	Heap Sort
100	100,0	19,5	45,2	100,0	53,7	48,0
500	100,0	4,5	10,2	100,0	12,5	8,8
1000	100,0	1,8	4,0	100,0	8,0	5,6
10000	100,0	0,2	0,5	100,0	1,0	0,5

**Nota: Com $N=10000$ (Tabela 3), o Shell Sort foi penalizado em esferas absolutas de dezenas de milhões de operações $\mathcal{O}(n^2)$, o que minimizou drasticamente a porcentagem visível dos demais métodos.*

4 Conclusões

A depuração estatística possibilita traçar três deduções consolidadas a respeito da arquitetura computacional dos métodos:

1. **A falha sistêmica do Shell Sort Original:** O intervalo padrão do Shell Sort ($n/2$) apresenta uma deficiência grave diante do arranjo Decrescente (Tabela 3). Por trabalhar exclusivamente através de potências de dois nos ciclos primários, ele isola posições pares de posições ímpares de forma ineficiente no começo do percurso. O resultado é o estrangulamento da CPU através da complexidade $\mathcal{O}(n^2)$, tornando o algoritmo 100% gargalo tanto em movimentação quanto em comparações neste cenário específico. Em vetores pré-organizados (Tabela 2), sua natureza ligada ao *Insertion Sort* acelera a estabilidade.
2. **O gargalo de memória do Merge Sort:** O mecanismo do *Merge Sort* provou ser rigorosamente limpo em comparações lógicas (permanecendo estável entre 40% e 49% relativos contra as outras arquiteturas). O calcanhar de Aquiles computacional exposto pelo relatório foi unicamente o peso de arrasto da movimentação $M(n)$. As

realocações dinâmicas de estruturas auxiliares custaram ao algoritmo a posição de líder absoluto em desperdício da memória nos Cenários 1 e 2 (100%).

3. **A quebra de ciclos lógicos no Heap Sort:** Diferentemente do *Merge Sort*, o *Heap Sort* não penaliza a memória por agir através da propriedade *in-place*. Essa contenção, contudo, tem custo na métrica $C(n)$. No Cenário 2 (vetor inicialmente crescente), o ato inicial de transmutar os dados ordenados na topologia de um max-heap (e depois arrastar as raízes máximas em cascata de substituição) corrompeu violentamente o alinhamento orgânico da lista original. Isso exigiu um grande número de checagens inúteis (rebaixamentos) que arremessaram o algoritmo para a margem de 100% de limite comparativo ineficaz.

Referências

- [1] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.